Table of Contents

E-Portfolio ***2***

https://github.com/luitoz/intelligent-agents-eportfolio 2

https://luitoz.github.io/intelligent-agents-eportfolio 2

Module Reflection ***3***

Appendices ***6***

A. Unit 1 Reflection **6**

B. Unit 2 Reflection **6**

C. Unit 3 Reflection **7**

D. Unit 4 Reflection **7**

E. Unit 5 Reflection **8**

F. Unit 6 Reflection **8**

G. Unit 7 Reflection **9**

H. Unit 8 Reflection **9**

I. Unit 9 Reflection **10**

J. Unit 11 Reflection **10**

K. ACL Dialogues **10**

L. Deep Learning in Action **12**

M. Team Project **13**

N. Individual Project **14**

O. Case Study on XAI **17**

P. Case Study on Safety Assurance **18**

Q. Case Study on Bias **18**

R. Collaborative Discussions **18**

S. Professional Skills Matrix **20**

T. Swot Analysis **21**

U. Action Plan (PDP) **22**

References ***23***

E-Portfolio

- GitHub repository containing artifacts developed during this module:

https://github.com/luitoz/intelligent-agents-eportfolio

-
- Web page:

https://luitoz.github.io/intelligent-agents-eportfolio

Module Reflection

This module developed my understanding of agent-based systems by comparing agent architectures, identifying the contexts in which they are effective, and examining the legal, social, and ethical implications of their deployment.

I explored symbolic agents, which use First-Order Logic and formal knowledge representations, alongside reactive agents, which were developed to address limitations in deliberative symbolic reasoning (Appendix B). Hybrid architectures such as TouringMachines demonstrated how deliberation and responsiveness can be combined, although they also showed that additional architectural complexity must be justified by practical requirements (Appendix D). I also examined Agent Communication Languages, interaction protocols, and semantic heterogeneity arising from different domain ontologies (Appendix K). These topics showed that interoperability and coordination remain difficult even when modern agents communicate mainly through natural language rather than standardized ACLs (Zhou *et al.*, 2025).

The module also traced the development of Natural Language Processing (NLP) from statistical methods to neural techniques such as Word2Vec (Appendix G). Studying constituency and dependency parsing clarified how syntactic relationships can be represented, while word embeddings demonstrated how semantic relationships can be encoded computationally (Appendix H). Artificial Neural Networks, deep learning, and Convolutional Neural Networks showed how models can learn hierarchical features automatically (Appendix I). However, I also recognized that this capacity must be balanced against requirements for data, computing resources, interpretability, and effective governance, particularly in Industry 4.0 applications (Sakurada and Leitão, 2020) and Smart Cities 4.0 (Appendix L).

Collaborative discussions helped me evaluate the trade-offs between centralized control and distributed autonomous-agent approaches, the risk of goal misalignment (Amodei *et al.*, 2016), and the safe adoption of agents in manufacturing (Appendix R). These activities moved my learning beyond technical descriptions because I had to compare competing designs, defend decisions, and consider how failures could affect users and organizations.

For the team project, I proposed the initial concept of transforming a broad research interest into a focused, feasible thesis topic (Appendix M). Through discussion and collective refinement, this idea evolved into the Academic Investigator, a Multi-Agent System supporting academic research tasks such as literature retrieval, synthesis, and reporting. I contributed to the agent-oriented design methodology and the iterative development approach of the system design. This experience taught me to treat an initial proposal as a starting point for collaboration rather than a solution that should remain

unchanged. Team feedback broadened the system's scope and encouraged me to evaluate my assumptions against alternative perspectives. I also learned that effective collaboration requires clear role allocation, constructive disagreement, shared documentation, and timely communication so that individual technical contributions remain aligned with the team's objectives and agreed system scope (Appendix F).

For the individual assessment, I implemented the system using a Belief-Desire-Intention architecture (Appendix N, Figure 8), Python, LangGraph, LangSmith, the Semantic Scholar API, and Qwen3 (Appendix N, Figure 4). Applying these technologies connected theoretical concepts with practical software-engineering decisions. Prompt engineering improved output consistency, while LangGraph's predefined schemas acted as guardrails that reduced off-topic responses (Appendix N, Figure 6). Nevertheless, these measures could not eliminate hallucinations or guarantee factual accuracy (Ji *et al.*, 2023), reinforcing the need for validation and human oversight.

One important decision was making paper retrieval fully deterministic (Appendix N, Figure 7). This improved predictability, controlled calls to the rate-limited Semantic Scholar API, and made behaviour easier to test. However, it created a hard-coded retrieval strategy that applied the same search logic regardless of the research question. This trade-off showed me that reliability can restrict adaptability. Future work could introduce an LLM-based retrieval skill that adapts queries across multiple academic databases while retaining explicit limits, transparent selection criteria, and measurable relevance.

Testing exposed another practical limitation. Because the Semantic Scholar API was rate-limited, I initially had to wait for availability before executing automated tests. I responded by implementing a mock service for integration testing and reserving the real API for functional testing (Appendix N, Figure 5). This reduced dependence on an external provider, accelerated development, and improved repeatability. More importantly, it taught me to design for testability by isolating external dependencies instead of treating service availability as an unavoidable development constraint.

Implementation also changed my view of the hybrid deployment I initially proposed, which combined a central LLM with locally deployed SLMs. In hindsight, this design would have required multiple models and additional computing resources while providing limited benefit, since both model types still depended on online paper retrieval and analysis. Rejecting unnecessary complexity was an important learning outcome. Future architectural choices should be based on demonstrated requirements and measurable value rather than novelty.

Responsible AI presented challenges. Bias could originate in individual papers, collections of literature, or API rankings that favour particular regions, languages, institutions, or publication venues (Kacperski *et al.*, 2024). Consequently, students

might receive an incomplete representation of a field and overlook relevant perspectives (Appendix Q). A stronger system would retrieve from multiple databases, disclose inclusion and ranking criteria, assess source diversity, and allow users to inspect and challenge recommendations. Explainability also requires attention because users need to trace system claims back to factual and auditable evidence (Appendix O).

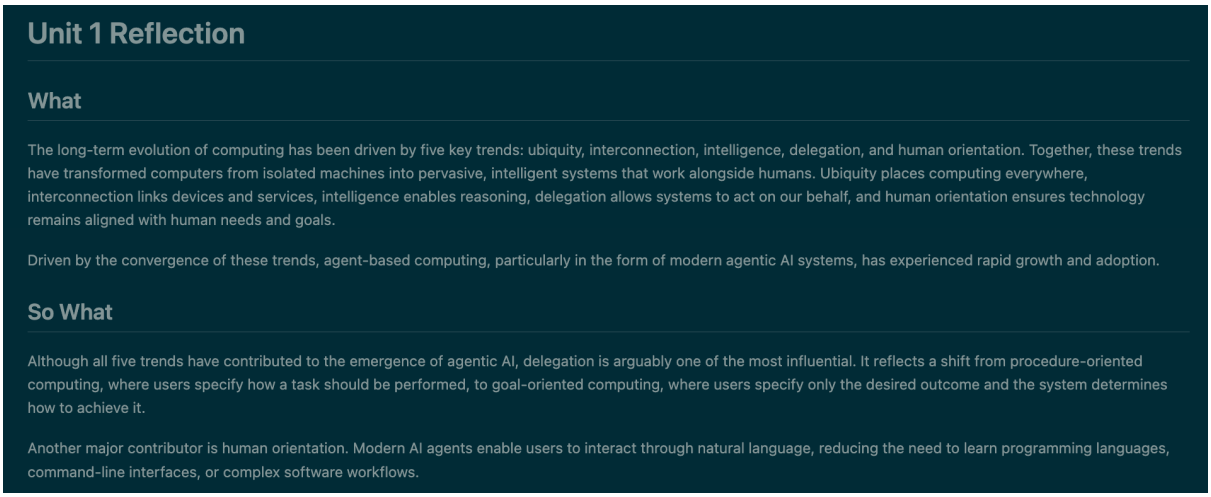
A fundamental takeaway is that modern AI often extends rather than replaces traditional approaches. Rule-based parsing and its resulting parse trees remain useful for modern NLP because they offer interpretability, lower data requirements, and easier debugging. Similarly, classical Machine Learning algorithms such as Gradient Boosted Trees may outperform neural networks on structured or limited (Grinsztajn, Oyallon and Varoquaux, 2022). These comparisons reinforced that the most fashionable technique is not necessarily the most suitable one.

Professionally, the project reinforced the importance of documenting design decisions, communicating limitations, respecting external service constraints, and distinguishing prototypes from production-ready systems. Reliable practice also requires monitoring dependencies like external APIs, LLMs, and network services, testing failure conditions, and avoiding claims unsupported by evaluation. In future projects, I will define quality measures earlier, assess ethical risks throughout development, involve users in evaluation, and ensure that AI outputs remain traceable and open to human review.

In conclusion, this module demonstrated that effective agent-based systems rely not only on sound architectural decisions but also on robust software engineering practices and responsible deployment. By integrating theoretical concepts from agent architectures, deep learning, and NLP with practical projects, I developed a critical understanding of the strengths, limitations, and trade-offs involved in building intelligent systems for domains such as manufacturing and academic research. Most importantly, I learned to evaluate technologies based on their suitability for a given problem rather than their novelty, while carefully considering reliability, transparency, ethical implications, and user needs.

Appendices

A. Unit 1 Reflection



Unit 1 Reflection

What

The long-term evolution of computing has been driven by five key trends: ubiquity, interconnection, intelligence, delegation, and human orientation. Together, these trends have transformed computers from isolated machines into pervasive, intelligent systems that work alongside humans. Ubiquity places computing everywhere, interconnection links devices and services, intelligence enables reasoning, delegation allows systems to act on our behalf, and human orientation ensures technology remains aligned with human needs and goals.

Driven by the convergence of these trends, agent-based computing, particularly in the form of modern agentic AI systems, has experienced rapid growth and adoption.

So What

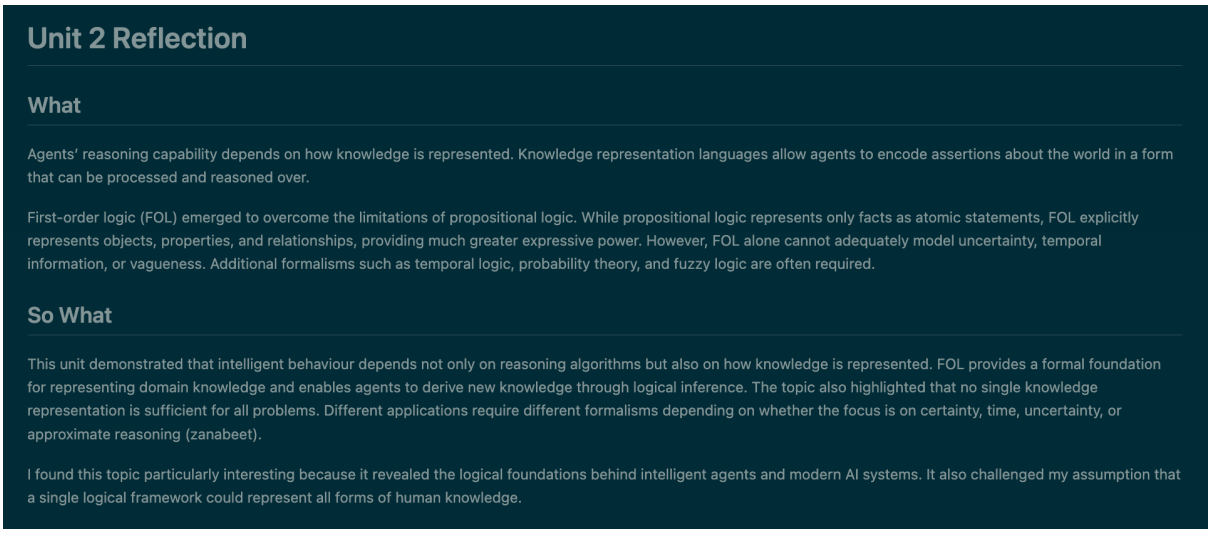
Although all five trends have contributed to the emergence of agentic AI, delegation is arguably one of the most influential. It reflects a shift from procedure-oriented computing, where users specify how a task should be performed, to goal-oriented computing, where users specify only the desired outcome and the system determines how to achieve it.

Another major contributor is human orientation. Modern AI agents enable users to interact through natural language, reducing the need to learn programming languages, command-line interfaces, or complex software workflows.

GitHub URL:

https://github.com/luitoz/intelligent-agents-eportfolio/blob/main/files/intelligent-agents/unit1/reflection.md

B. Unit 2 Reflection



Unit 2 Reflection

What

Agents' reasoning capability depends on how knowledge is represented. Knowledge representation languages allow agents to encode assertions about the world in a form that can be processed and reasoned over.

First-order logic (FOL) emerged to overcome the limitations of propositional logic. While propositional logic represents only facts as atomic statements, FOL explicitly represents objects, properties, and relationships, providing much greater expressive power. However, FOL alone cannot adequately model uncertainty, temporal information, or vagueness. Additional formalisms such as temporal logic, probability theory, and fuzzy logic are often required.

So What

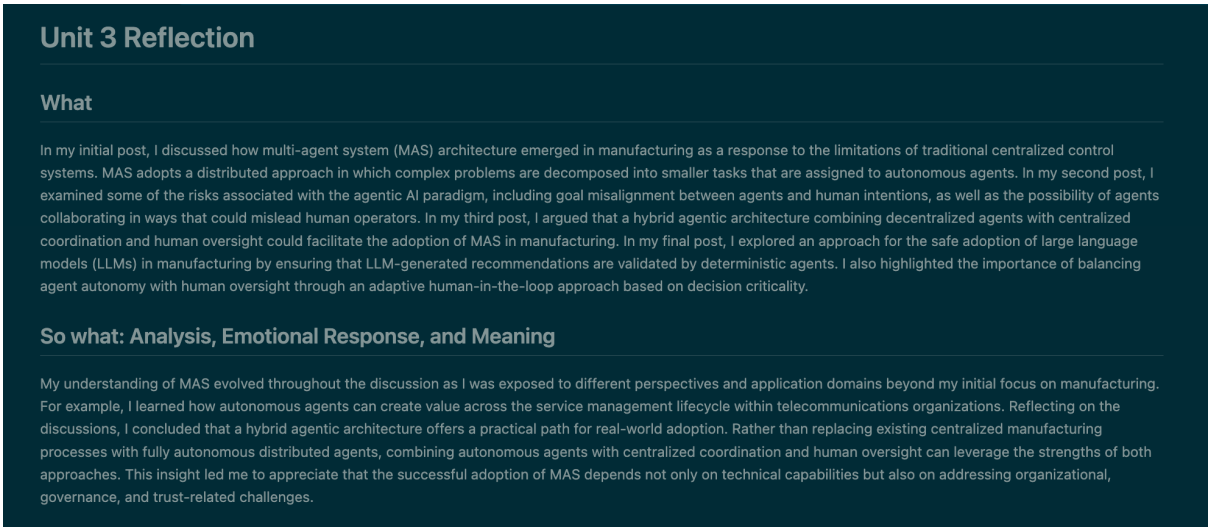
This unit demonstrated that intelligent behaviour depends not only on reasoning algorithms but also on how knowledge is represented. FOL provides a formal foundation for representing domain knowledge and enables agents to derive new knowledge through logical inference. The topic also highlighted that no single knowledge representation is sufficient for all problems. Different applications require different formalisms depending on whether the focus is on certainty, time, uncertainty, or approximate reasoning (zanabeet).

I found this topic particularly interesting because it revealed the logical foundations behind intelligent agents and modern AI systems. It also challenged my assumption that a single logical framework could represent all forms of human knowledge.

GitHub URL:

https://github.com/luitoz/intelligent-agents-eportfolio/blob/main/files/intelligent-agents/unit2/reflection.md

C. Unit 3 Reflection



Unit 3 Reflection

What

In my initial post, I discussed how multi-agent system (MAS) architecture emerged in manufacturing as a response to the limitations of traditional centralized control systems. MAS adopts a distributed approach in which complex problems are decomposed into smaller tasks that are assigned to autonomous agents. In my second post, I examined some of the risks associated with the agentic AI paradigm, including goal misalignment between agents and human intentions, as well as the possibility of agents collaborating in ways that could mislead human operators. In my third post, I argued that a hybrid agentic architecture combining decentralized agents with centralized coordination and human oversight could facilitate the adoption of MAS in manufacturing. In my final post, I explored an approach for the safe adoption of large language models (LLMs) in manufacturing by ensuring that LLM-generated recommendations are validated by deterministic agents. I also highlighted the importance of balancing agent autonomy with human oversight through an adaptive human-in-the-loop approach based on decision criticality.

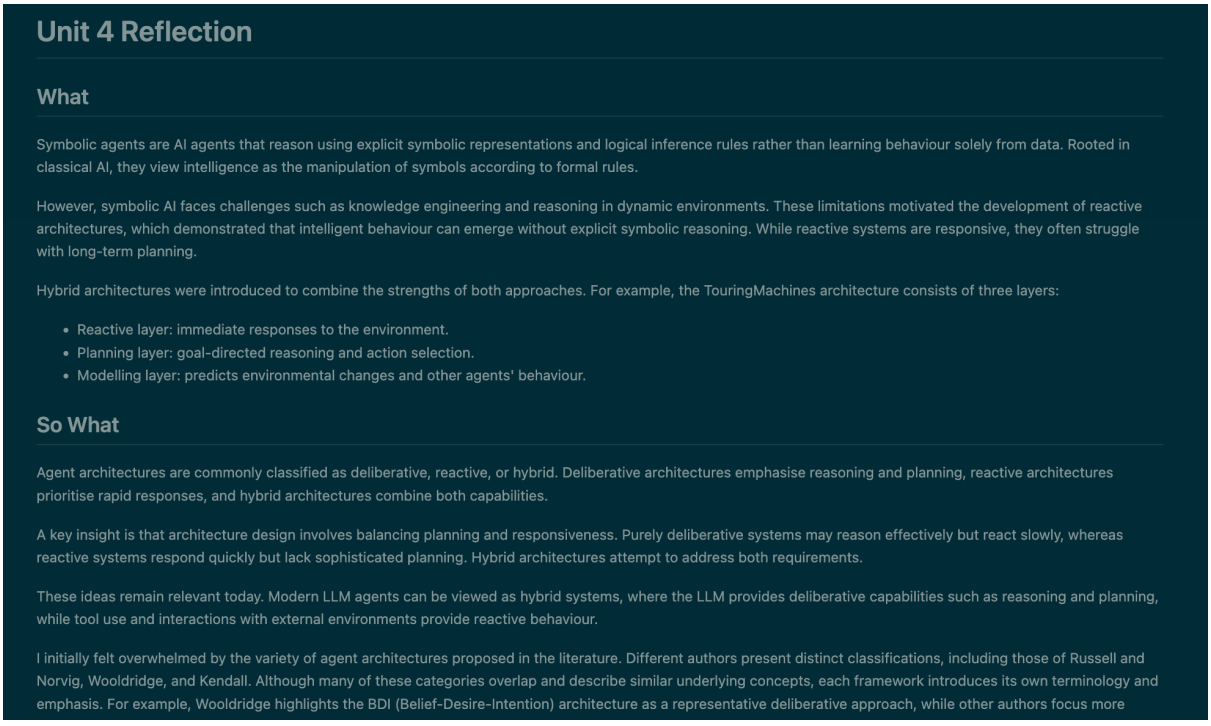
So what: Analysis, Emotional Response, and Meaning

My understanding of MAS evolved throughout the discussion as I was exposed to different perspectives and application domains beyond my initial focus on manufacturing. For example, I learned how autonomous agents can create value across the service management lifecycle within telecommunications organizations. Reflecting on the discussions, I concluded that a hybrid agentic architecture offers a practical path for real-world adoption. Rather than replacing existing centralized manufacturing processes with fully autonomous distributed agents, combining autonomous agents with centralized coordination and human oversight can leverage the strengths of both approaches. This insight led me to appreciate that the successful adoption of MAS depends not only on technical capabilities but also on addressing organizational, governance, and trust-related challenges.

GitHub URL:

https://github.com/luitoz/intelligent-agents-eportfolio/blob/main/files/intelligent-agents/unit3/reflection.md

D. Unit 4 Reflection



Unit 4 Reflection

What

Symbolic agents are AI agents that reason using explicit symbolic representations and logical inference rules rather than learning behaviour solely from data. Rooted in classical AI, they view intelligence as the manipulation of symbols according to formal rules.

However, symbolic AI faces challenges such as knowledge engineering and reasoning in dynamic environments. These limitations motivated the development of reactive architectures, which demonstrated that intelligent behaviour can emerge without explicit symbolic reasoning. While reactive systems are responsive, they often struggle with long-term planning.

Hybrid architectures were introduced to combine the strengths of both approaches. For example, the TouringMachines architecture consists of three layers:

- Reactive layer: immediate responses to the environment.
- Planning layer: goal-directed reasoning and action selection.
- Modelling layer: predicts environmental changes and other agents' behaviour.

So What

Agent architectures are commonly classified as deliberative, reactive, or hybrid. Deliberative architectures emphasise reasoning and planning, reactive architectures prioritise rapid responses, and hybrid architectures combine both capabilities.

A key insight is that architecture design involves balancing planning and responsiveness. Purely deliberative systems may reason effectively but react slowly, whereas reactive systems respond quickly but lack sophisticated planning. Hybrid architectures attempt to address both requirements.

These ideas remain relevant today. Modern LLM agents can be viewed as hybrid systems, where the LLM provides deliberative capabilities such as reasoning and planning, while tool use and interactions with external environments provide reactive behaviour.

I initially felt overwhelmed by the variety of agent architectures proposed in the literature. Different authors present distinct classifications, including those of Russell and Norvig, Wooldridge, and Kendall. Although many of these categories overlap and describe similar underlying concepts, each framework introduces its own terminology and emphasis. For example, Wooldridge highlights the BDI (Belief-Desire-Intention) architecture as a representative deliberative approach, while other authors focus more

GitHub URL:

https://github.com/luitoz/intelligent-agents-eportfolio/blob/main/files/intelligent-agents/unit4/reflection.md

E. Unit 5 Reflection

Unit 5 Reflection

What

Agent communication is grounded in speech act theory, which views communication as consisting of two components: a performative that expresses the communicative intention (e.g., inform, request, or agree) and a propositional content that represents the information being communicated. Based on this theory, Agent Communication Languages (ACLs) emerged to support cooperation and interoperability in multi-agent systems. For example, FIPA ACL standardizes agent communication through a set of communicative acts, message structures, and interaction protocols. These protocols are particularly valuable in highly cooperative settings such as distributed planning, coordination, and task allocation.

A major challenge in agent communication is semantic heterogeneity, where agents may use different ontologies to represent domain knowledge. This issue, known as ontology alignment, is not addressed directly by ACLs, which generally assume that communicating agents share sufficient semantic understanding to interpret message content correctly. Instead, ontology alignment is typically handled through ontology matching and mapping techniques, sometimes complemented by negotiation-based interactions in which agents exchange assertions, objections, acceptances, and rejections to establish unambiguous semantic correspondences between concepts.

So what

Although Agent Communication Languages such as FIPA ACL originated within the symbolic agent paradigm and are commonly used with deliberative architectures, they are not restricted to symbolic agents. ACLs specify communication acts and interaction protocols rather than internal reasoning mechanisms. Consequently, reactive, learning-based, and LLM-driven agents can also employ ACL-style communication, provided they can interpret the performatives and message content. Modern LLM-based multi-agent systems often implement ACL concepts implicitly through natural-language interactions rather than formal communication languages.

It is fascinating how the challenges and solutions of agent communication have evolved from classical multi-agent systems to modern agentic AI. The need for a common communication standard remains as relevant today as it was in early agent research. While formal Agent Communication Languages (ACLs), such as FIPA ACL, were designed primarily for symbolic and deliberative agents, they are rarely used in contemporary LLM-based agent systems. Instead, modern approaches increasingly rely on natural language as the primary medium of interaction between agents and between humans and agents.

GitHub URL:

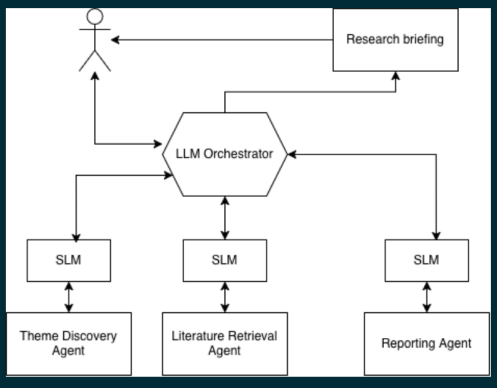
https://github.com/luitoz/intelligent-agents-eportfolio/blob/main/files/intelligent-agents/unit5/reflection.md

F. Unit 6 Reflection

Unit 6 Reflection

What

- Team members introduced themselves, their backgrounds, locations (especially time zones), and preferred communication platforms.
- The first meeting focused on understanding the project brief and identifying an appropriate approach, including the agent classification and architecture to use. Team members were then assigned to prepare proposals containing a problem statement and a high-level agent architecture.
- My proposal consisted of a MAS system that helps postgraduate students to plan their thesis, transforming a broad area of interest into a focused, feasible, and academically valuable thesis topic. The proposed system should identify current research trends, discover gaps in existing literature, review academic publications, and formulate relevant research questions.
- This proposal's main design decision was to use a Large Language Model (LLM)-based agent as a central orchestrator, coordinating multiple Small Language Model (SLM)-based agents responsible for domain-specific tasks (Farahani, Khan and Wuest, 2026). This architecture aims to reduce latency by minimizing communication overhead to a central LLM and to improve resource utilization by distributing load between the central LLM and lightweight SLMs.

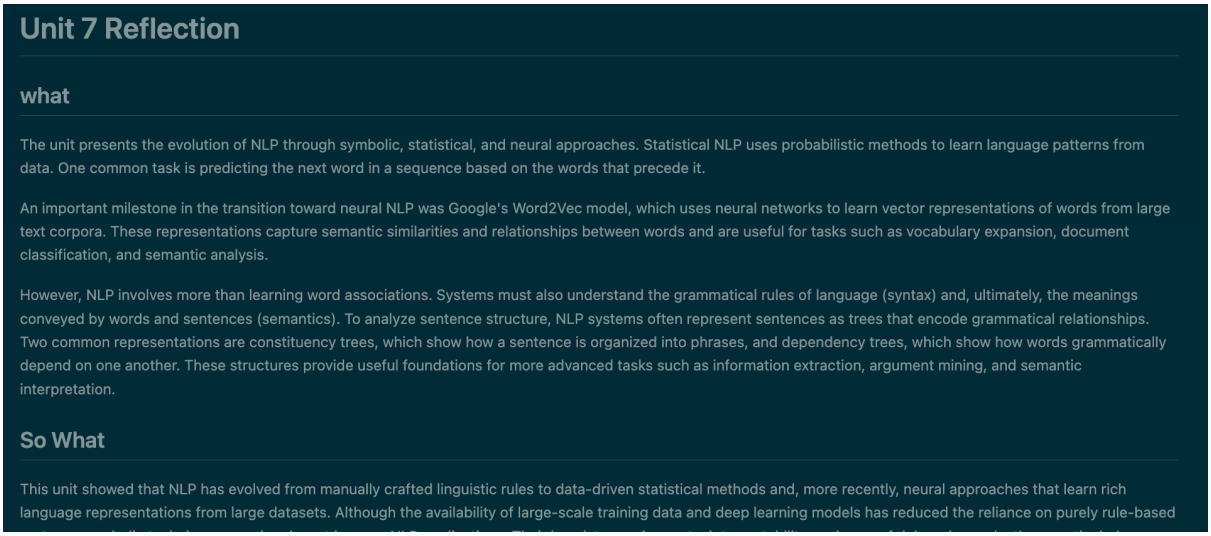


- During the second meeting, we reviewed and compared the proposals, particularly those developed by Isaac and me, and selected one to move forward with. We

GitHub URL:

https://github.com/luitoz/intelligent-agents-eportfolio/blob/main/files/intelligent-agents/unit6/reflection.md

G. Unit 7 Reflection



Unit 7 Reflection

what

The unit presents the evolution of NLP through symbolic, statistical, and neural approaches. Statistical NLP uses probabilistic methods to learn language patterns from data. One common task is predicting the next word in a sequence based on the words that precede it.

An important milestone in the transition toward neural NLP was Google's Word2Vec model, which uses neural networks to learn vector representations of words from large text corpora. These representations capture semantic similarities and relationships between words and are useful for tasks such as vocabulary expansion, document classification, and semantic analysis.

However, NLP involves more than learning word associations. Systems must also understand the grammatical rules of language (syntax) and, ultimately, the meanings conveyed by words and sentences (semantics). To analyze sentence structure, NLP systems often represent sentences as trees that encode grammatical relationships. Two common representations are constituency trees, which show how a sentence is organized into phrases, and dependency trees, which show how words grammatically depend on one another. These structures provide useful foundations for more advanced tasks such as information extraction, argument mining, and semantic interpretation.

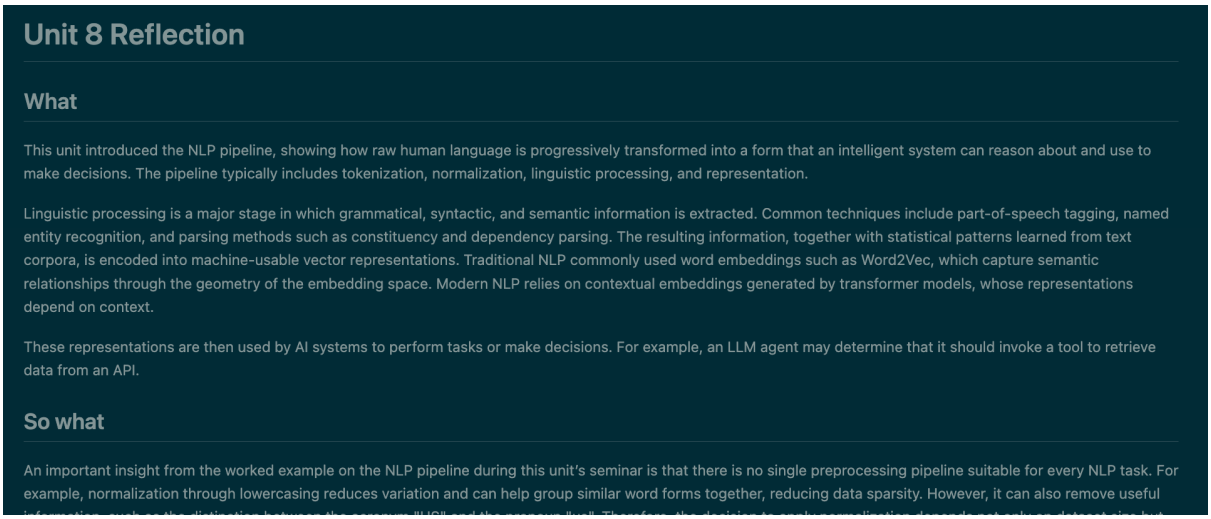
So What

This unit showed that NLP has evolved from manually crafted linguistic rules to data-driven statistical methods and, more recently, neural approaches that learn rich language representations from large datasets. Although the availability of large-scale training data and deep learning models has reduced the reliance on purely rule-based

GitHub URL:

https://github.com/luitoz/intelligent-agents-eportfolio/blob/main/files/intelligent-agents/unit7/reflection.md

H. Unit 8 Reflection



Unit 8 Reflection

What

This unit introduced the NLP pipeline, showing how raw human language is progressively transformed into a form that an intelligent system can reason about and use to make decisions. The pipeline typically includes tokenization, normalization, linguistic processing, and representation.

Linguistic processing is a major stage in which grammatical, syntactic, and semantic information is extracted. Common techniques include part-of-speech tagging, named entity recognition, and parsing methods such as constituency and dependency parsing. The resulting information, together with statistical patterns learned from text corpora, is encoded into machine-usable vector representations. Traditional NLP commonly used word embeddings such as Word2Vec, which capture semantic relationships through the geometry of the embedding space. Modern NLP relies on contextual embeddings generated by transformer models, whose representations depend on context.

These representations are then used by AI systems to perform tasks or make decisions. For example, an LLM agent may determine that it should invoke a tool to retrieve data from an API.

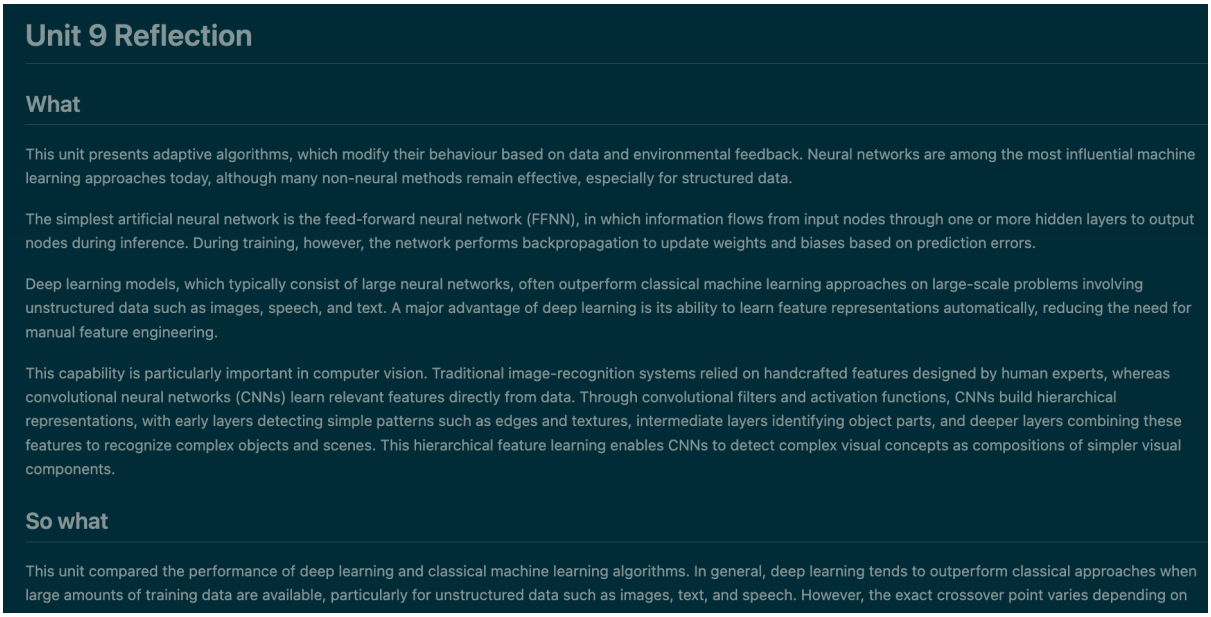
So what

An important insight from the worked example on the NLP pipeline during this unit's seminar is that there is no single preprocessing pipeline suitable for every NLP task. For example, normalization through lowercasing reduces variation and can help group similar word forms together, reducing data sparsity. However, it can also remove useful

GitHub URL:

https://github.com/luitoz/intelligent-agents-eportfolio/blob/main/files/intelligent-agents/unit8/reflection.md

I. Unit 9 Reflection



Unit 9 Reflection

What

This unit presents adaptive algorithms, which modify their behaviour based on data and environmental feedback. Neural networks are among the most influential machine learning approaches today, although many non-neural methods remain effective, especially for structured data.

The simplest artificial neural network is the feed-forward neural network (FFNN), in which information flows from input nodes through one or more hidden layers to output nodes during inference. During training, however, the network performs backpropagation to update weights and biases based on prediction errors.

Deep learning models, which typically consist of large neural networks, often outperform classical machine learning approaches on large-scale problems involving unstructured data such as images, speech, and text. A major advantage of deep learning is its ability to learn feature representations automatically, reducing the need for manual feature engineering.

This capability is particularly important in computer vision. Traditional image-recognition systems relied on handcrafted features designed by human experts, whereas convolutional neural networks (CNNs) learn relevant features directly from data. Through convolutional filters and activation functions, CNNs build hierarchical representations, with early layers detecting simple patterns such as edges and textures, intermediate layers identifying object parts, and deeper layers combining these features to recognize complex objects and scenes. This hierarchical feature learning enables CNNs to detect complex visual concepts as compositions of simpler visual components.

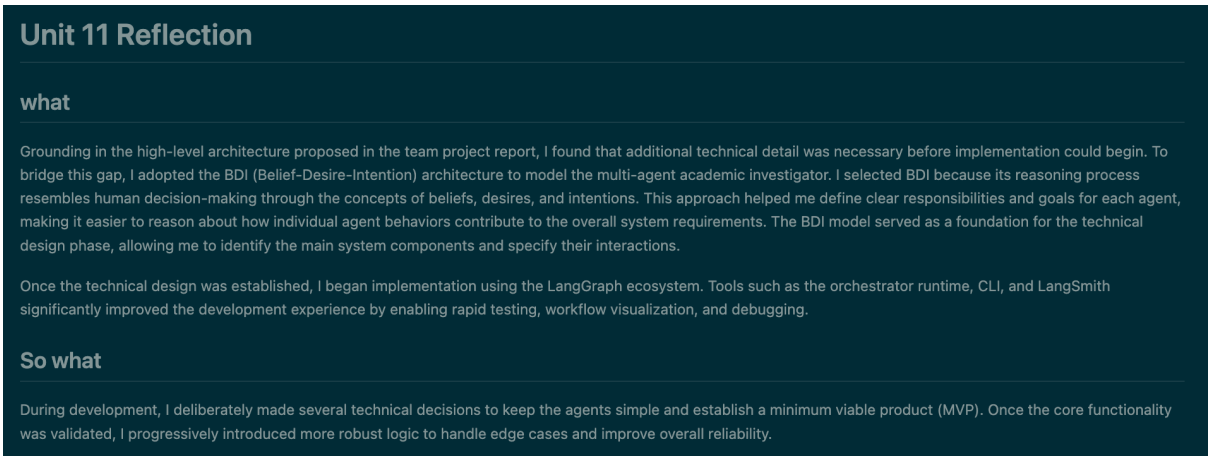
So what

This unit compared the performance of deep learning and classical machine learning algorithms. In general, deep learning tends to outperform classical approaches when large amounts of training data are available, particularly for unstructured data such as images, text, and speech. However, the exact crossover point varies depending on

GitHub URL:

https://github.com/luitoz/intelligent-agents-eportfolio/blob/main/files/intelligent-agents/unit9/reflection.md

J. Unit 11 Reflection



Unit 11 Reflection

what

Grounding in the high-level architecture proposed in the team project report, I found that additional technical detail was necessary before implementation could begin. To bridge this gap, I adopted the BDI (Belief-Desire-Intention) architecture to model the multi-agent academic investigator. I selected BDI because its reasoning process resembles human decision-making through the concepts of beliefs, desires, and intentions. This approach helped me define clear responsibilities and goals for each agent, making it easier to reason about how individual agent behaviors contribute to the overall system requirements. The BDI model served as a foundation for the technical design phase, allowing me to identify the main system components and specify their interactions.

Once the technical design was established, I began implementation using the LangGraph ecosystem. Tools such as the orchestrator runtime, CLI, and LangSmith significantly improved the development experience by enabling rapid testing, workflow visualization, and debugging.

So what

During development, I deliberately made several technical decisions to keep the agents simple and establish a minimum viable product (MVP). Once the core functionality was validated, I progressively introduced more robust logic to handle edge cases and improve overall reliability.

GitHub URL:

https://github.com/luitoz/intelligent-agents-eportfolio/blob/main/files/intelligent-agents/unit11/reflection.md

K. ACL Dialogues

This exercise involved identifying the goals and responsibilities of two agents and analysing their interactions. Since both agents operate in the same domain, they share a common ontology, including entities such as Product, Feature, and Quantity.

KQML dialogues use Knowledge Interchange Format (KIF) as the content language, whereas FIPA dialogues use FIPA Semantic Language (FIPA-SL).

In KQML, ask-one and tell support information queries, sorry handles errors, and achieve communicates procurement requests. In FIPA, query-ref and inform support queries, failure handles errors, and request initiates procurement actions. Unlike KQML's standalone performatives, FIPA organises interactions through the FIPA-Query and FIPA-Request protocols, which explicitly define both success and failure outcomes.

Exchange 3 — unknown product (error handling)

```
(ask-one
  :sender alice
  :receiver bob
  :content (stock-level television-80in ?quantity)
  :reply-with q3
  :language KIF
  :ontology warehouse-stock-ontology)

(sorry
  :sender bob
  :receiver alice
  :content (unknown-product television-80in)
  :in-reply-to q3
  :language KIF
  :ontology warehouse-stock-ontology)
```

Exchange 4 — procurement decision

```
(achieve
  :sender alice
  :receiver bob
  :content (procure-decision television-50in ?decision)
  :reply-with q4
  :language KIF
  :ontology warehouse-stock-ontology)

(tell
  :sender bob
  :receiver alice
  :content (procure-decision television-50in proceed)
  :in-reply-to q4
  :language KIF
  :ontology warehouse-stock-ontology)
```

Figure 1. KQML dialogue

Exchange 3 — unknown product (failure branch)

```
(query-ref
  :sender alice
  :receiver bob
  :content (stock-level television-80in)
  :reply-with q3
  :language fipa-sl
  :ontology warehouse-stock-ontology
  :protocol fipa-query
  :conversation-id conv1)

(failure
  :sender bob
  :receiver alice
  :content (unknown-product television-80in)
  :in-reply-to q3
  :language fipa-sl
  :ontology warehouse-stock-ontology
  :protocol fipa-query
  :conversation-id conv1)
```

FIPA Request Protocol (fipa-request)

Exchange 4 — procurement decision

```
(request
  :sender alice
  :receiver bob
  :content (action bob (decide-procurement television-50in))
  :reply-with q4
  :language fipa-sl
  :ontology warehouse-stock-ontology
  :protocol fipa-request
  :conversation-id conv1)


(inform-done
  :sender bob
  :receiver alice
  :content (done
             (action bob (decide-procurement television-50in))
             (= (procure-decision television-50in) proceed))
  :in-reply-to q4
  :language fipa-sl
  :ontology warehouse-stock-ontology
  :protocol fipa-request
  :conversation-id conv1)
```

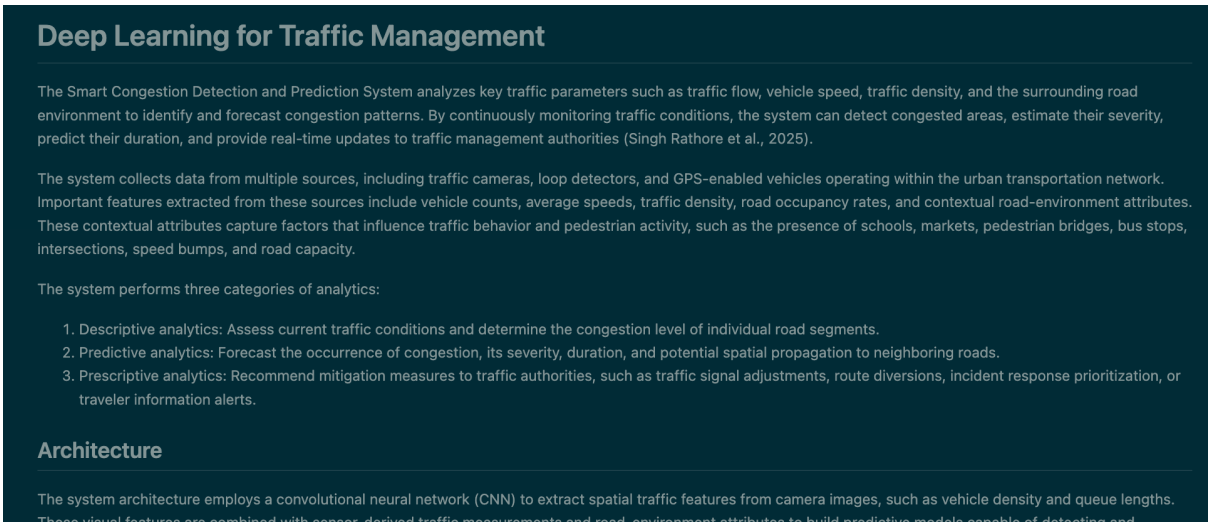
Figure 2. FIPA dialogue

GitHub URL:

https://github.com/luitoz/intelligent-agents-eportfolio/blob/main/files/intelligent-agents/unit6/ACL-dialogues.md

L. Deep Learning in Action

Brief: You are required to research an application of Deep Learning that you think is going to have an impact on society.



Deep Learning for Traffic Management

The Smart Congestion Detection and Prediction System analyzes key traffic parameters such as traffic flow, vehicle speed, traffic density, and the surrounding road environment to identify and forecast congestion patterns. By continuously monitoring traffic conditions, the system can detect congested areas, estimate their severity, predict their duration, and provide real-time updates to traffic management authorities (Singh Rathore et al., 2025).

The system collects data from multiple sources, including traffic cameras, loop detectors, and GPS-enabled vehicles operating within the urban transportation network. Important features extracted from these sources include vehicle counts, average speeds, traffic density, road occupancy rates, and contextual road-environment attributes. These contextual attributes capture factors that influence traffic behavior and pedestrian activity, such as the presence of schools, markets, pedestrian bridges, bus stops, intersections, speed bumps, and road capacity.

The system performs three categories of analytics:

1. Descriptive analytics: Assess current traffic conditions and determine the congestion level of individual road segments.
2. Predictive analytics: Forecast the occurrence of congestion, its severity, duration, and potential spatial propagation to neighboring roads.
3. Prescriptive analytics: Recommend mitigation measures to traffic authorities, such as traffic signal adjustments, route diversions, incident response prioritization, or traveler information alerts.

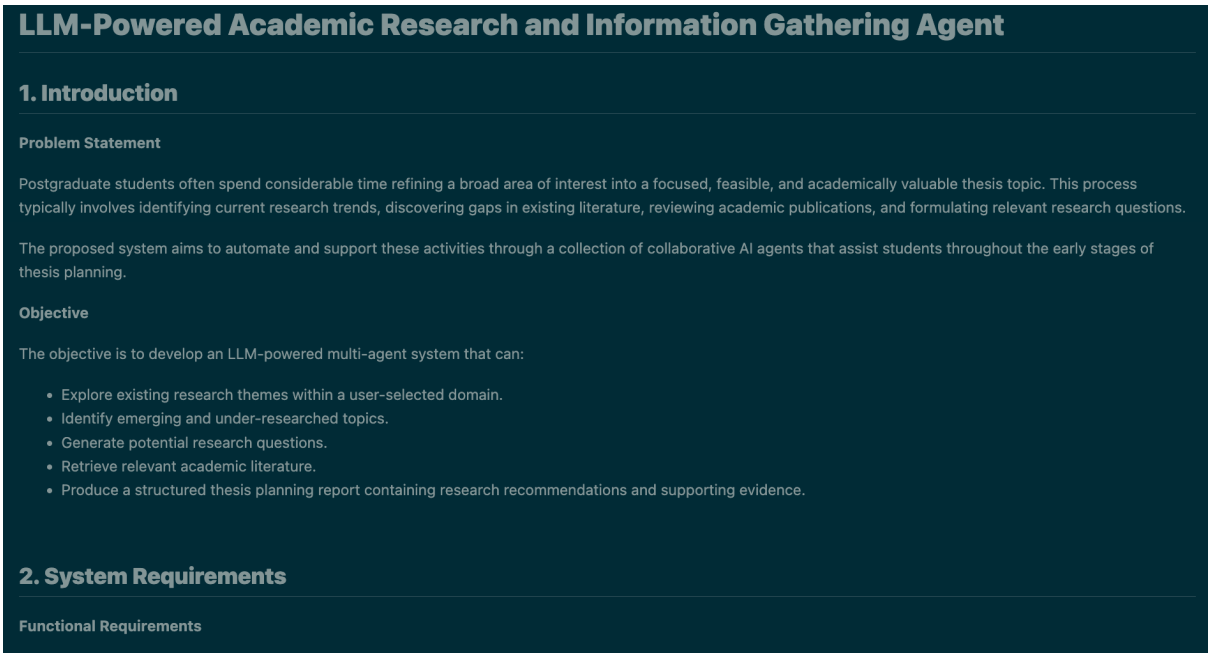
Architecture

The system architecture employs a convolutional neural network (CNN) to extract spatial traffic features from camera images, such as vehicle density and queue lengths.

GitHub URL:

https://github.com/luitoz/intelligent-agents-eportfolio/blob/main/files/intelligent-agents/unit10/deep-learning-in-action.md

M. Team Project



LLM-Powered Academic Research and Information Gathering Agent

1. Introduction

Problem Statement

Postgraduate students often spend considerable time refining a broad area of interest into a focused, feasible, and academically valuable thesis topic. This process typically involves identifying current research trends, discovering gaps in existing literature, reviewing academic publications, and formulating relevant research questions.

The proposed system aims to automate and support these activities through a collection of collaborative AI agents that assist students throughout the early stages of thesis planning.

Objective

The objective is to develop an LLM-powered multi-agent system that can:

- Explore existing research themes within a user-selected domain.
- Identify emerging and under-researched topics.
- Generate potential research questions.
- Retrieve relevant academic literature.
- Produce a structured thesis planning report containing research recommendations and supporting evidence.

2. System Requirements

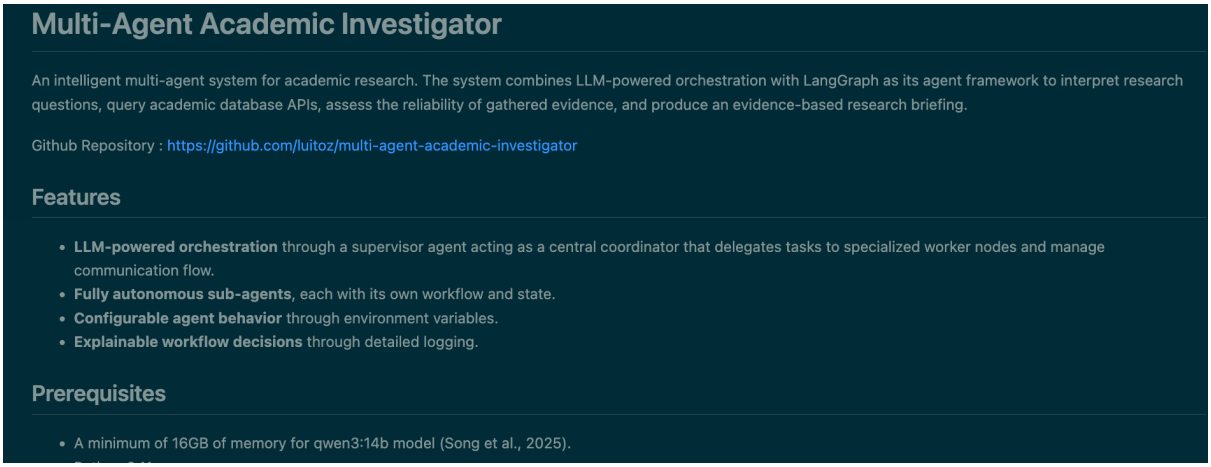
Functional Requirements

Figure 3. Initial proposal for the MAS project

GitHub URL:

https://github.com/luitoz/intelligent-agents-eportfolio/blob/main/files/intelligent-agents/unit6/team-project/team-project-proposal.md

N. Individual Project



Multi-Agent Academic Investigator

An intelligent multi-agent system for academic research. The system combines LLM-powered orchestration with LangGraph as its agent framework to interpret research questions, query academic database APIs, assess the reliability of gathered evidence, and produce an evidence-based research briefing.

Github Repository : https://github.com/luitoz/multi-agent-academic-investigator

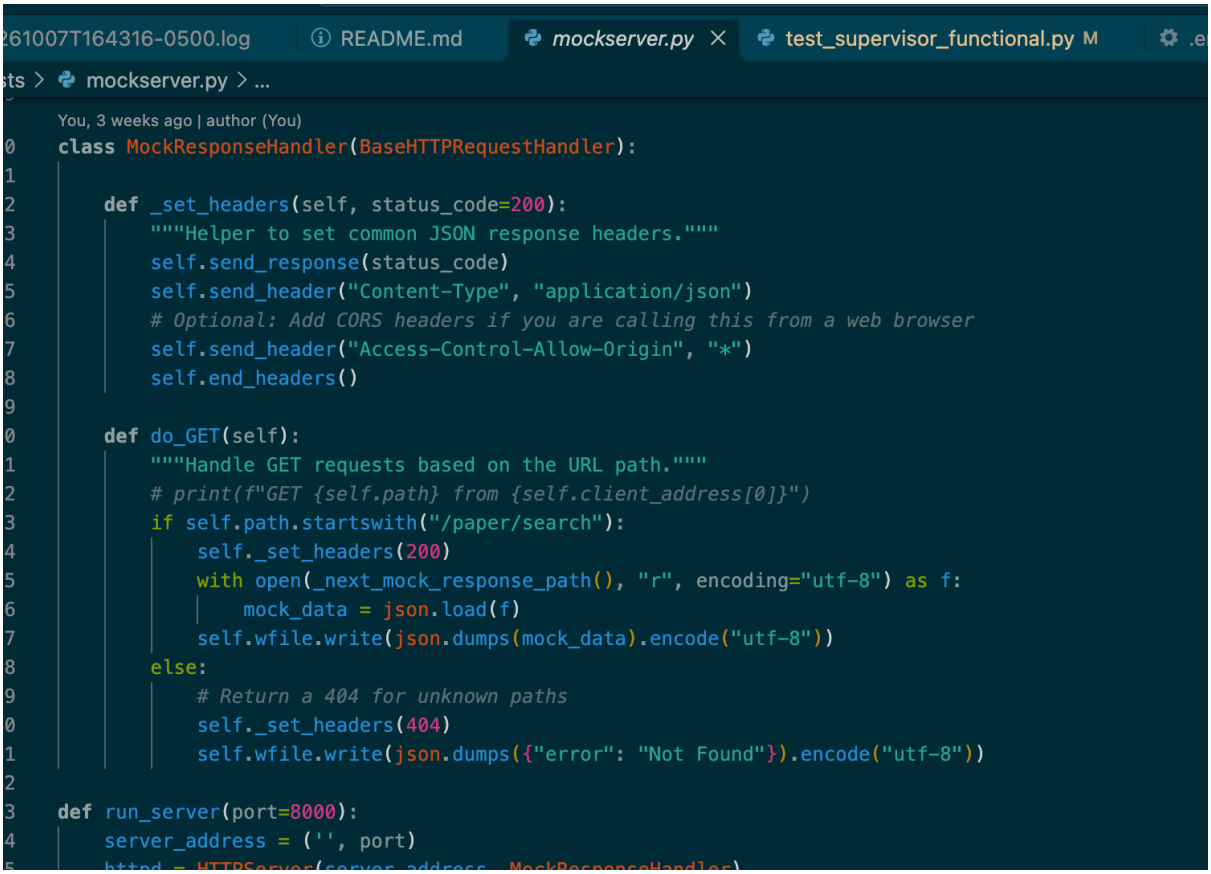
Features

- **LLM-powered orchestration** through a supervisor agent acting as a central coordinator that delegates tasks to specialized worker nodes and manage communication flow.
- **Fully autonomous sub-agents**, each with its own workflow and state.
- **Configurable agent behavior** through environment variables.
- **Explainable workflow decisions** through detailed logging.

Prerequisites

- A minimum of 16GB of memory for qwen3:14b model (Song et al., 2025).

Figure 4. MAS project's README file



261007T164316-0500.log | README.md | mockserver.py | test_supervisor_functional.py M

sts > mockserver.py > ...

You, 3 weeks ago | author (You)

```python
class MockResponseHandler(BaseHTTPRequestHandler):

    def _set_headers(self, status_code=200):
        """Helper to set common JSON response headers."""
        self.send_response(status_code)
        self.send_header("Content-Type", "application/json")
        # Optional: Add CORS headers if you are calling this from a web browser
        self.send_header("Access-Control-Allow-Origin", "*")
        self.end_headers()

    def do_GET(self):
        """Handle GET requests based on the URL path."""
        # print(f"GET {self.path} from {self.client_address[0]}")
        if self.path.startswith("/paper/search"):
            self._set_headers(200)
            with open(_next_mock_response_path(), "r", encoding="utf-8") as f:
                mock_data = json.load(f)
            self.wfile.write(json.dumps(mock_data).encode("utf-8"))
        else:
            # Return a 404 for unknown paths
            self._set_headers(404)
            self.wfile.write(json.dumps({"error": "Not Found"}).encode("utf-8"))

def run_server(port=8000):
    server_address = ('', port)
```

Figure 5. Mock for the Semantic Scholar API

```python
    You, 3 weeks ago | author (You)
class ResearchFindings(BaseModel):
    supporting_evidence: str = Field(
        description="Evidence across the papers that supports the research question, with Harvard-style citations. "
        "2-3 sentences."
    )
    conflicting_evidence: str = Field(
        description="Evidence across the papers that conflicts or disagrees, with Harvard-style citations. "
        "2-3 sentences."
    )
    main_conclusions: str = Field(
        description="The overall conclusions drawn from synthesizing the evidence. 2-3 sentences."
    )
    methodology: str = Field(
        description="The methodologies used across the papers, compared where relevant. 2-3 sentences."
    )
    limitations: str = Field(
        description="Key limitations of the evidence, including gaps, inconsistencies, or inconclusive areas. "
        "2-3 sentences."
    )
```

```python
def synthesize_findings(state: SupervisorState, config: RunnableConfig) -> dict:
    """Step 4: once the gathered papers are deemed reliable, synthesize their findings into insights."""
    papers = state.get("papers") or []
    if not papers:
        raise RuntimeError("No papers were available to analyze.")

    papers_text = "\n\n".join(
        f"Title: {paper.title}\nAuthors: {_format_authors(paper.authors)}\nYear: {paper.year}\n"
        f"Abstract: {paper.abstract}"
        for paper in papers
    )
    logger.info(f"Analyzing {len(papers)} reliable papers and identifying insights")
    findings_model = model.with_structured_output(ResearchFindings)      You, 3 weeks ago • added a mock server fo
    output = findings_model.invoke(
        [
            {"role": "system", "content": SYNTHESIZE_FINDINGS_PROMPT.format(research_question=state["request"])},
            {"role": "user", "content": papers_text},
        ],
        config=config,
    )
```

Figure 6. LangGraph schema to constrain LLM output



Figure 7. Paper retrieval deterministic logic exposed as an LLM tool

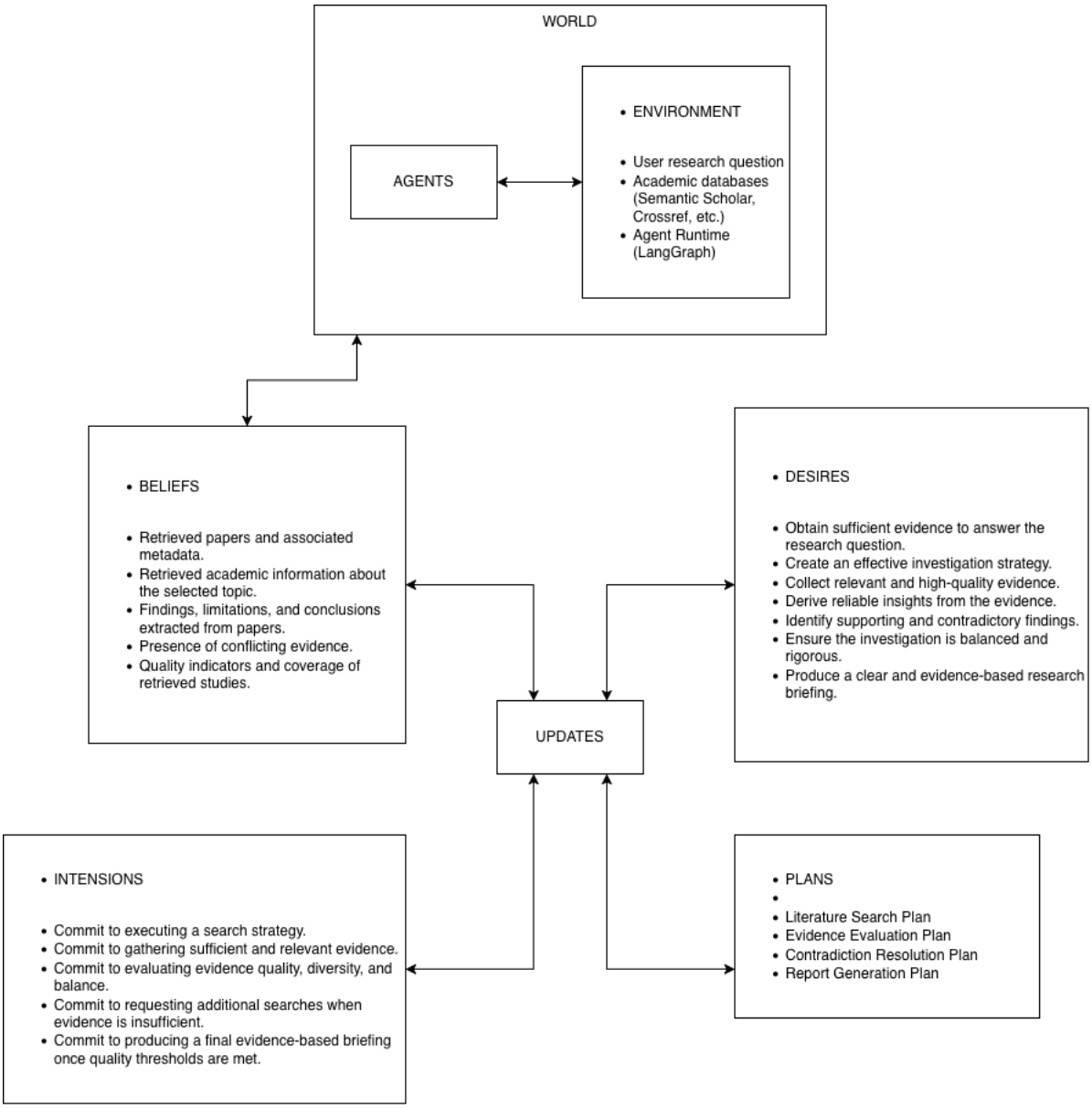


Figure 8. BDI Architecture for the MAS

O. Case Study on XAI

Reflection on XAI

What

The Multi-Agent Academic Investigator uses LangGraph, which explicitly defines state, nodes, edges, and tools, making the agent workflow transparent.

The evidence analysis agent uses a deterministic screening process for retrieved papers. Only the aggregate verdict (reliable, questionable, or unreliable) is kept in the graph state; the individual check results and per-paper scores are written to the log file only.

LLM outputs (objectives, search dimensions, synthesis, and briefing) are included in the graph state, but the system does not capture a rationale or show how alternative objectives, queries, or interpretations were considered.

LLM is instructed to generate claims only from gathered papers, but there is no explicit mapping from each claim to source-paper excerpts that can be verified.

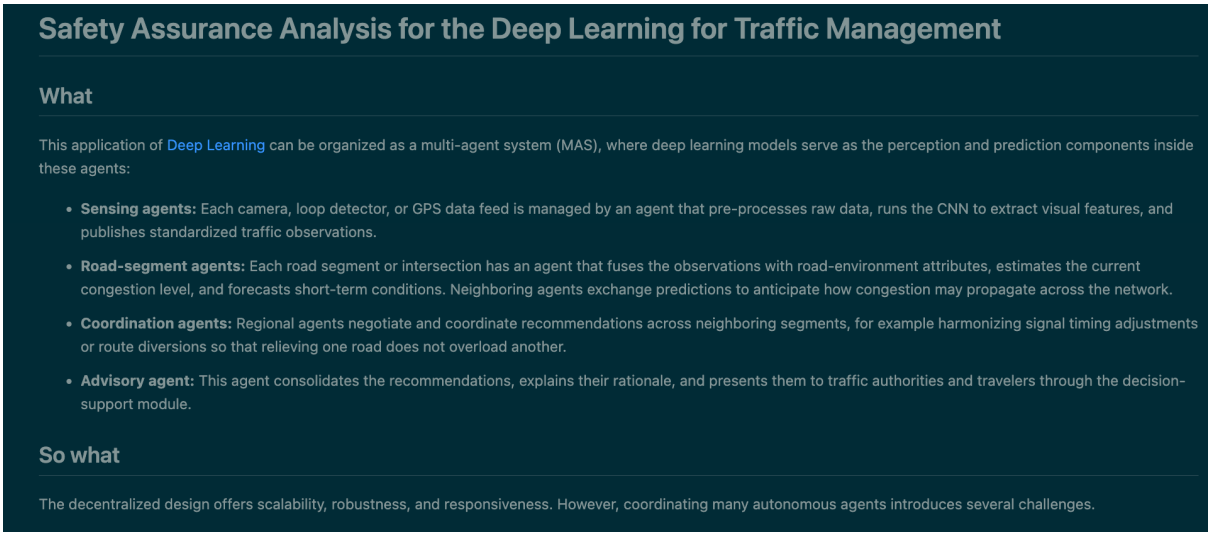
So what

As a result, a user or evaluator cannot currently see why a paper was rejected, reproduce a search, or verify that a reported claim follows from the papers cited.

GitHub URL:

https://github.com/luitoz/intelligent-agents-eportfolio/blob/main/files/intelligent-agents/unit12/xai_reflection.md

P. Case Study on Safety Assurance



Safety Assurance Analysis for the Deep Learning for Traffic Management

What

This application of Deep Learning can be organized as a multi-agent system (MAS), where deep learning models serve as the perception and prediction components inside these agents:

- **Sensing agents:** Each camera, loop detector, or GPS data feed is managed by an agent that pre-processes raw data, runs the CNN to extract visual features, and publishes standardized traffic observations.
- **Road-segment agents:** Each road segment or intersection has an agent that fuses the observations with road-environment attributes, estimates the current congestion level, and forecasts short-term conditions. Neighboring agents exchange predictions to anticipate how congestion may propagate across the network.
- **Coordination agents:** Regional agents negotiate and coordinate recommendations across neighboring segments, for example harmonizing signal timing adjustments or route diversions so that relieving one road does not overload another.
- **Advisory agent:** This agent consolidates the recommendations, explains their rationale, and presents them to traffic authorities and travelers through the decision-support module.

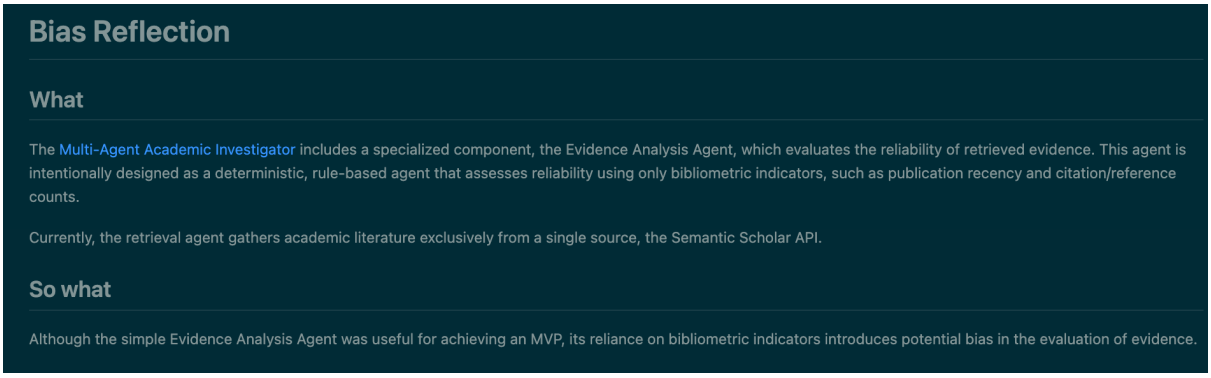
So what

The decentralized design offers scalability, robustness, and responsiveness. However, coordinating many autonomous agents introduces several challenges.

GitHub URL:

https://github.com/luitoz/intelligent-agents-eportfolio/blob/main/files/intelligent-agents/unit12/safety_assurance_reflection.md

Q. Case Study on Bias



Bias Reflection

What

The Multi-Agent Academic Investigator includes a specialized component, the Evidence Analysis Agent, which evaluates the reliability of retrieved evidence. This agent is intentionally designed as a deterministic, rule-based agent that assesses reliability using only bibliometric indicators, such as publication recency and citation/reference counts.

Currently, the retrieval agent gathers academic literature exclusively from a single source, the Semantic Scholar API.

So what

Although the simple Evidence Analysis Agent was useful for achieving an MVP, its reliance on bibliometric indicators introduces potential bias in the evaluation of evidence.

GitHub URL:

https://github.com/luitoz/intelligent-agents-eportfolio/blob/main/files/intelligent-agents/unit12/bias_reflection.md

R. Collaborative Discussions

- Initial post:
- https://github.com/luitoz/intelligent-agents-eportfolio/blob/main/files/intelligent-agents/unit1/initial-post.pdf
-
- Peer Response 1:
- https://github.com/luitoz/intelligent-agents-eportfolio/blob/main/files/intelligent-agents/unit2/peer-response.pdf
-
- Peer Response 2:
- https://github.com/luitoz/intelligent-agents-eportfolio/blob/main/files/intelligent-agents/unit2/peer-response-2.pdf
-
- Summary post:
- https://github.com/luitoz/intelligent-agents-eportfolio/blob/main/files/intelligent-agents/unit3/summary-post.pdf

S. Professional Skills Matrix

COMPETENCY	SKILL	SKILL LEVEL	EVIDENCE
PROFESSIONAL	Time Management	Proficient	Planned and completed weekly machine learning labs, coding activities, seminars, and assessments to deadlines, balancing independent study with practical model development and evaluation tasks.
	Critical Thinking and Analysis	Proficient	Evaluated algorithm suitability for different datasets, compared model performance metrics, interpreted results critically, and assessed model strengths, limitations, and bias.
	Communication and Literacy Skills	Expert	Produced written reflections, technical reports, and presentations explaining machine learning processes, model outputs, and evidence-based conclusions for both technical and non-technical audiences.
	Research	Proficient	Investigated machine learning methodologies, explored datasets systematically, compared alternative models, and applied evidence-based approaches to solve data-driven problems.
IT AND DIGITAL	Python	Proficient	Used Python and Jupyter Notebook to import, clean, preprocess, visualise, analyse, and model data using machine learning techniques.
	GitHub	Proficient	Managed machine learning projects, version-controlled code, documented workflows, and maintained reproducible notebooks and scripts within a GitHub portfolio.
TECHNOLOGIES	Python	Proficient	Applied Python libraries including Pandas, NumPy, Scikit-learn, Matplotlib, and Seaborn for data preparation, model implementation, evaluation, and visualisation.
	Jupyter Notebook	Proficient	Developed and documented end-to-end machine learning workflows, including preprocessing, model training, validation, and reporting.
	HTML	Expert	6+ years of experience creating web pages and supporting front-end development projects.
	LangGraph	Proficient	Designed and implemented agent-based workflows using LangGraph, defining agent interactions, routing logic, state management, and tool integration to coordinate complex tasks.
	Large Language Models (LLMs)	Proficient	Applied foundation models within intelligent agent systems for natural language understanding, reasoning, information retrieval, and task execution.
NUMERACY	Machine Learning & Data Analysis	Trained	Applied data preprocessing techniques, feature engineering, model training, cross-validation, performance evaluation, and interpretation of metrics such as accuracy, precision, recall, F1-score, and RMSE.
SOCIAL / TEAMWORK	Academic Teamwork	Trained	Participated in seminars and collaborative discussions, explaining model selections, evaluating peer approaches, providing constructive feedback, and reflecting on alternative solutions.
APPLICATION	Problem-Solving	Proficient	Addressed real-world machine learning challenges including missing data, class imbalance, overfitting, underfitting, feature selection issues, and model optimisation through iterative experimentation.
	Agent-Based Problem Solving	Proficient	Developed agent-oriented solutions for complex problems by integrating reasoning, planning, communication, and external tools within autonomous systems.
LEGAL & ETHICAL	Ethical Awareness	Trained	Recognised ethical considerations in machine learning, including algorithmic bias, fairness, transparency, data privacy, responsible use of AI, and the limitations of predictive models when making decisions.

T. Swot Analysis

Columna1	HELPFUL	HAMPER
INTERNAL	**Stregths.**	**Weaknesses**
	15+ years industry: 5 as a Software Architect, running development teams, mostly in the exigent banking industry	Working on filling the gap between professional practice and academic study after a long time without undertaking formal studies
	Undergraduate: Systems Engineering	Tendency to focus on technical delivery rather than a more holistic approach
	Highly self-motivated and success driven. Autonomous learner with a great emphasis on details	Because work hard, I need to also allocate time for mental and physical health
EXTERNAL	**Opportunities**	**Threads**
	My role within a leading-edge technology firm provides a unique vantage point to translate academic concepts into high-impact business solutions.	Managing high-volume coursework alongside other responsibilities occasionally challenges my work-life harmony
	The greenfield nature of AI in my region provides a unique opening to experiment freely and build a future-ready ecosystem from the ground up	Occasional time zone differences prevent me from attending key academic events, such as lectures and seminars
	Seeking to leverage and expand my collaborative skills by fostering reciprocal learning and shared insight within a dynamic team	Course material has some errors, leading to confusion

U. Action Plan (PDP)

WHAT DO I WANT / NEED TO LEARN?	WHAT DO I HAVE TO DO TO ACHIEVE THIS?	WHAT RESOURCES OR SUPPORT WILL I NEED?	HOW WILL I MEASURE SUCCESS?	TARGET DATES FOR REVIEW AND COMPLETION	EVIDENCE OF DEVELOPMENT
Develop understanding of intelligent agent architectures	Study Reactive, Deliberative, Hybrid, and BDI architectures and analyse their suitability for different applications	Lecture notes, module materials, Wooldridge textbook, academic papers	Ability to explain and justify the selection of appropriate agent architectures for different problems	Mid-module review; end-of-module completion	Coursework notes, architecture comparisons, reflective learning journal
Strengthen multi-agent system design skills	Design agents that communicate, coordinate, and cooperate to achieve shared goals	Module labs, case studies, FIPA communication materials, workshop activities	Successful design and analysis of multi-agent interactions and coordination strategies	Mid-module	Agent designs, workflow diagrams, seminar activities
Improve agent orchestration and development skills	Implement agent workflows using frameworks such as LangGraph and integrate external tools and APIs	LangGraph documentation, coding exercises, GitHub repositories	Functional agent workflows demonstrating reasoning, routing, and tool use	Throughout module	Code repositories, workflow implementations, demonstration outputs
Develop competence in large language model integration	Configure, evaluate, and compare LLMs such as Qwen3 for intelligent agent applications	Model documentation, Hugging Face resources, practical exercises	Ability to justify model selection and evaluate strengths and limitations for agent tasks	End of implementation unit	Evaluation notes, model comparisons, experimentation records
Evaluate agent performance and reliability	Analyse agent behaviour, reasoning quality, coordination effectiveness, and failure scenarios	Research papers, testing frameworks, seminar guidance	Ability to identify limitations, explain failures, and propose improvements	Throughout module	Testing reports, evaluation summaries, reflective analyses
Communicate intelligent agent solutions effectively	Produce clear documentation, architecture diagrams, presentations, and critical reflections	Presentation guidance, peer feedback, diagramming tools	Clear and professional explanations of agent systems and design decisions	Before assessment submission	Final report, architecture diagrams, presentation slides
Develop independent and reflective learning skills in agentic AI	Manage project work independently, explore emerging agent frameworks, and critically reflect on learning	GitHub, academic literature, online documentation, self-study resources	Consistent progress and ability to solve implementation challenges with minimal supervision	Weekly review; end of module	GitHub commit history, project logs, reflective journal
Develop awareness of ethical and responsible agent deployment	Investigate issues such as bias, hallucinations, transparency, accountability, privacy, and human oversight	Responsible AI guidelines, module materials, research papers	Ability to critically evaluate ethical risks and propose mitigation strategies	End of module	Ethics discussion, risk assessments, reflective commentary

References

Amodei, D. *et al.* (2016) ‘Concrete Problems in AI Safety’. arXiv. Available at: https://doi.org/10.48550/arXiv.1606.06565.

Grinsztajn, L., Oyallon, E. and Varoquaux, G. (2022) ‘Why do tree-based models still outperform deep learning on typical tabular data?’, *Advances in Neural Information Processing Systems*. Curran Associates, Inc., pp. 507–520. Available at: https://doi.org/10.52202/068431-0037.

Ji, Z. *et al.* (2023) ‘Survey of Hallucination in Natural Language Generation’, *ACM Computing Surveys*, 55(12), pp. 1–38. Available at: https://doi.org/10.1145/3571730.

Kacperski, C. *et al.* (2024) ‘Examining bias perpetuation in academic search engines: an algorithm audit of Google and Semantic Scholar’, *First Monday* [Preprint]. Available at: https://doi.org/10.5210/fm.v29i11.13730.

Sakurada, L. and Leitão, P. (2020) ‘Multi-Agent Systems to Implement Industry 4.0 Components’, *2020 IEEE Conference on Industrial Cyberphysical Systems (ICPS)*. *2020 IEEE Conference on Industrial Cyberphysical Systems (ICPS)*, pp. 21–26. Available at: https://doi.org/10.1109/ICPS48405.2020.9274745.

Zhou, P. *et al.* (2025) ‘Why do AI agents communicate in human language?’ arXiv. Available at: https://doi.org/10.48550/arXiv.2506.02739.